

The background is a solid light blue. A large, horizontal, pink brushstroke-like shape is centered behind the text. Above the text, there are five small circles: two purple, two dark blue, and one light pink.

mood *Match* *Reads*

Lydia Morgan
Alaa Abdelrahman
Labiba Rahman Suhaila

the problem

When you're a reader, making your way through your TBR (to be read) can be a challenge.

CHOICE

Too many books to choose from

TIMING

Time wasted trying to find the perfect book

MOOD

Hard to match a book to current mood

mood *Match* *Reads*

Where your mood meets your next read.



*Tailored
recommendations
in seconds*

*Powered by
online book
metadata*

***Less searching,
more reading***

How it Works:

1

Share your mood

Users enter preferences on book type; mood, genre, author

Type

Genre

Author

Mood

2

Matching...

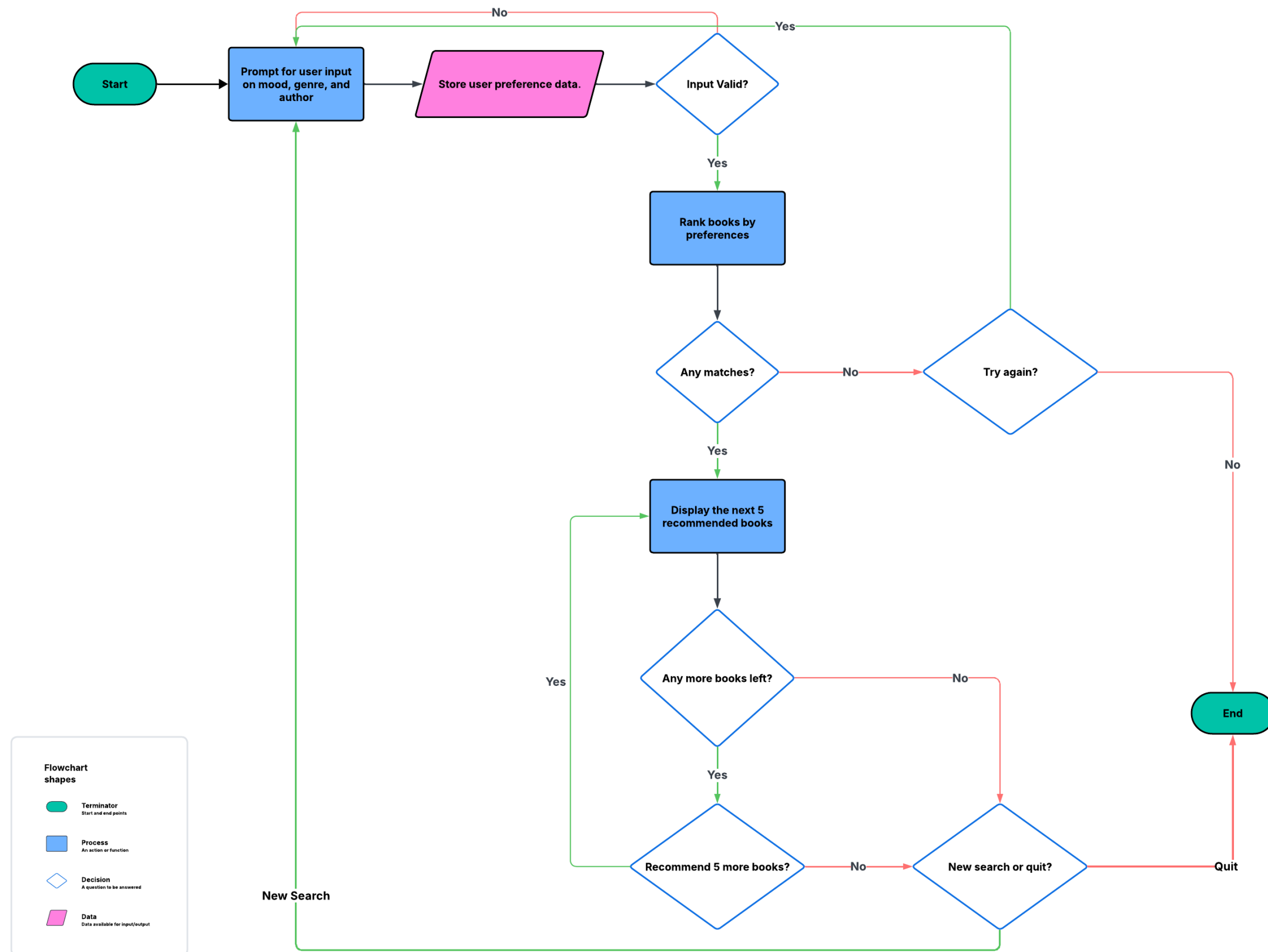
Preferences are matched with online book metadata

3

Your matches!

Get tailored recommendations instantly.

flowchart



building the library

BOOK COLLECTION

30 popular books with real ratings
Pulled from openlibrary.org
by Claude AI

BOOK FORMAT

5 fields per book: title, author, genre,
mood, rating

ORGANIZED BY GENRE

Books are grouped by genre in our
BOOKS list

Building the Library

The books were organized into lists including the title, author, genre, mood, and rating

```
title = book[0]
book_author = book[1]
book_genre = book[2]
book_mood = book[3]
rating = book[4]
```

```
BOOKS = [

# Romance
["Pride and Prejudice", "Jane Austen", "romance", "romantic", 4.28],
["The Notebook", "Nicholas Sparks", "romance", "sad", 4.16],
["Beach Read", "Emily Henry", "romance", "happy", 4.00],
["It Ends with Us", "Colleen Hoover", "romance", "emotional", 4.16],
["The Hating Game", "Sally Thorne", "romance", "happy", 3.97],

# Fantasy
["The Hobbit", "J.R.R. Tolkien", "fantasy", "adventurous", 4.28],
["Harry Potter", "J.K. Rowling", "fantasy", "magical", 4.47],
["A Court of Thorns and Roses", "Sarah J. Maas", "fantasy", "romantic", 4.18],
["The Name of the Wind", "Patrick Rothfuss", "fantasy", "magical", 4.55],
["Mistborn", "Brandon Sanderson", "fantasy", "adventurous", 4.49],

# Mystery
["Gone Girl", "Gillian Flynn", "mystery", "dark", 4.10],
["The Silent Patient", "Alex Michaelides", "mystery", "suspenseful", 4.18],
["And Then There Were None", "Agatha Christie", "mystery", "suspenseful", 4.30],
["The Girl on the Train", "Paula Hawkins", "mystery", "dark", 3.92],

# Sci-fi
["Dune", "Frank Herbert", "scifi", "adventurous", 4.26],
["Project Hail Mary", "Andy Weir", "scifi", "thoughtful", 4.52],
["1984", "George Orwell", "scifi", "dark", 4.19],
["The Martian", "Andy Weir", "scifi", "adventurous", 4.41],

# Classic
["To Kill a Mockingbird", "Harper Lee", "classic", "thoughtful", 4.28],
["The Great Gatsby", "F. Scott Fitzgerald", "classic", "sad", 3.93],
["Little Women", "Louisa May Alcott", "classic", "cozy", 4.19],
["Jane Eyre", "Charlotte Bronte", "classic", "romantic", 4.15],

# Self-help
["Atomic Habits", "James Clear", "selfhelp", "uplifting", 4.36],
["The 7 Habits of Highly Effective People", "Stephen Covey", "selfhelp", "uplifting", 4.16],

# Horror
["The Shining", "Stephen King", "horror", "dark", 4.27],
["Mexican Gothic", "Silvia Moreno-Garcia", "horror", "dark", 3.66],

# Young Adult
["The Fault in Our Stars", "John Green", "ya", "sad", 4.15],
["Six of Crows", "Leigh Bardugo", "ya", "adventurous", 4.46],
["The Hunger Games", "Suzanne Collins", "ya", "adventurous", 4.34],
["Eleanor & Park", "Rainbow Rowell", "ya", "romantic", 4.06],

]
```


the code

DATA SETUP

Lists set up the books, moods, and genres

FUNCTIONS

Functions break the code into clear steps

LOOPS

Loops keep the program running until the user is done

welcome message

PRINT WELCOME

Shows the welcome banner at startup

PRINT LIST

Shows the moods or genres in a
numbered list

FIRST IMPRESSION

These functions form the user's first
impression of the app

Welcome Message

print_welcome:
Prints a welcome message at the start of the program

```
## FUNCTION: print_welcome ##
def print_welcome():
    print("=" * 50 + "\n          MOOD MATCH READS\n          Where your mood meets your next read.\n" + "=" * 50)
    print("\nHi there! Welcome to Mood Match Reads.\nTell us a bit about what you are in the mood for\nand we will find your next favorite book.\n")
```

print_list(options):
prints a list of options with numbers next to them

```
## FUNCTION: print_list ##
def print_list(options):
    for i in range(len(options)):
        print("  " + str(i + 1) + ". " + options[i])
```

user input

CHOOSE

Pick a mood and/or genre by number or name. User has option to skip by pressing Enter

AUTHOR

Type any author name or press Enter to skip

INPUT VALIDATION

Each function re-prompts when the user gives bad input

User Input

ask_for_choice:

asks the user for input and shows a list of options (mood)

```
## FUNCTION: ask_for_choice ##
def ask_for_choice(question, options):
    print(question)
    print_list(options)

    while True:
        answer = input("Choose a number, a word, or press Enter to skip: ")
        answer = answer.strip().lower()

        if answer == "":
            return ""

        if answer.isdigit():
            number = int(answer)
            if number >= 1 and number <= len(options):
                return options[number - 1]
            else:
                print("Invalid input. Please pick a number between 1 and " + str(len(options)) + ".")

        elif answer in options:
            return answer

        else:
            print("Invalid input. Please type a number from the list, "
                  "the name of an option, or press Enter to skip.")
```

ask_for_author:

asks the user for input for a favorite author (optional)

```
## FUNCTION: ask_for_author ##
def ask_for_author():
    while True:
        answer = input("\nFavorite author? (type a name or press Enter to skip): ")
        answer = answer.strip()
        if answer == "":
            return ""
        if answer.isdigit():
            print("Invalid input. Please type an author's name or press Enter to skip.")
        else:
            return answer
```

ask_yes_no:

asks the user for input for a (genre)

```
## FUNCTION: ask_yes_no ##
def ask_yes_no(question):
    while True:
        answer = input(question).strip().lower()
        if answer == "yes" or answer == "y":
            return True
        elif answer == "no" or answer == "n":
            return False
        else:
            print("Invalid input. Please type 'yes' or 'no'.")
```

ranking the books

SCORING

Score each book by mood, genre, and author matches

SORTING

Sort by score so the best matches come first

PAGED RESULTS

Show 5 books at a time so the user is not overwhelmed

Ranking the Books

rank_books:

Uses a scoring system to rank books by mood, genre and author, within a for loop

```
## FUNCTION: rank_books ##
def rank_books(mood, genre, author):
    scored_books = []

    for book in BOOKS:
        title = book[0]
        book_author = book[1]
        book_genre = book[2]
        book_mood = book[3]
        rating = book[4]

        score = 0

        if mood != "" and book_mood == mood:
            score = score + 3

        if genre != "" and book_genre == genre:
            score = score + 2

        if author != "":
            if author.lower() in book_author.lower():
                score = score + 3

        if score > 0:
            score = score + rating
            scored_books.append([score, title, book_author, book_genre, rating])

    scored_books.sort()
    scored_books = scored_books[::-1]

    return scored_books
```

show_books:

shows a list of 5 books in descending order to the user with the best match first

```
## FUNCTION: show_books ##
def show_books(scored_books, start_index):
    print("\n" + "-" * 50 + "\n  Your matches:\n" + "-" * 50)

    end_index = start_index + 5
    if end_index > len(scored_books):
        end_index = len(scored_books)

    for i in range(start_index, end_index):
        book = scored_books[i]
        title = book[1]
        author = book[2]
        genre = book[3]
        rating = book[4]
        print("\n  " + str(i + 1) + ". " + title + "\n    by " + author + "\n    Genre: " + genre + "    Rating: " + str(rating) + " / 5")

    print()
```

Main Program

SCORING

Score each book by mood, genre, and author matches

SORTING

Sort by score so the best matches come first

PAGED RESULTS

Show 5 books at a time so the user is not overwhelmed

Main Program

```
## FUNCTION: main program ##
def main():
    print_welcome()

    while True:

        # ---- STEP 1: Ask for preferences ----
        print("Step 1: Tell us your preferences\n(You can skip any of these by pressing Enter)\n")

        mood = ask_for_choice("\nWhat mood are you in?", MOODS)
        genre = ask_for_choice("\nWhat genre do you want?", GENRES)
        author = ask_for_author()

        # "Input Valid?" diamond - the user must answer at least one question
        if mood == "" and genre == "" and author == "":
            print("\nInvalid input. Please answer at least one question\nso we can find a book for you.\n")
        else:
            # ---- STEP 2: Rank the books ----
            scored_books = rank_books(mood, genre, author)

            # If no books matched, ask if they want to try again
            if len(scored_books) == 0:
                print("\nNo books matched your preferences. Try different ones!\n")
                if ask_yes_no("Try a new search? (yes/no): ") == False:
                    print("\n" + "=" * 50 + "\n  Thanks for using Mood Match Reads!\n  Happy reading!\n" + "=" * 50 + "\n")
                    return
```

```
            else:
                # ---- STEP 3: Show 5 books at a time ----
                # Start at the beginning of the list
                start_index = 0
                # Flag to keep the loop running
                showing_books = True

                while showing_books:
                    # Display the next 5 books starting at the current index
                    show_books(scored_books, start_index)
                    # Move the index forward by 5 for the next batch
                    start_index = start_index + 5

                    # Check if there are no more books left to show
                    if start_index >= len(scored_books):
                        # Notify the user and stop the loop
                        print("That's all the books we found!")
                        showing_books = False
                    else:
                        # "More recommendations?" diamond - ask if they want the next 5
                        answer = ask_yes_no("Want to see 5 more? (yes/no): ")
                        if answer == False:
                            # User said no - stop the loop
                            showing_books = False
```

```
                # ---- STEP 4: New search or quit? ----
                print("\nWhat would you like to do next?\n  1. Start a new search with different preferences\n  2. Quit Mood Match Reads")
                while True:
                    choice = input("Pick 1 or 2: ").strip()
                    if choice == "1":
                        print()
                        break
                    elif choice == "2":
                        print("\n" + "=" * 50 + "\n  Thanks for using Mood Match Reads!\n  Happy reading!\n" + "=" * 50)
                        return
                    else:
                        print("Invalid input. Please type 1 or 2.")

main()
```

why this matters?

EASIER TO CHOOSE

Reduces decision fatigue

SAVES TIME

So you can get back to reading!

A PERFECT MATCH

Personalized recommendation tailored to your mood



Demo

Time



Thank *You!*

Lydia Morgan
Alaa Abdelrahman
Labiba Rahman Suhaila

References

www.claude.ai